Chapter 4 recursos de séries temporais

▶ Carregar bibliotecas e configurações comuns

▶ Carregar bibliotecas adicionais

O `tsfeatures` pacote desenvolvido pela Nixtla permite calcular diversas características a partir de dados de séries temporais. É o equivalente em Python do pacote `time` do `tsfeatures` R.

Já vimos algumas características de séries temporais. Por exemplo, as autocorrelações discutidas na Seção 2.8 podem ser consideradas características de uma série temporal — são resumos numéricos calculados a partir da série. Outra característica que vimos no capítulo anterior foi a estimativa de Guerrero do parâmetro de transformação de Box-Cox — novamente, trata-se de um número calculado a partir de uma série temporal.

Podemos calcular diversas características em várias séries temporais e usá-las para explorar as propriedades dessas séries. Neste capítulo, examinaremos algumas características que se mostraram úteis na exploração de séries temporais e como elas podem ser usadas para descobrir informações interessantes sobre seus dados. Usaremos o turismo trimestral australiano como exemplo (discutido anteriormente na Seção 2.5).

4.1 Algumas estatísticas simples

Qualquer resumo numérico calculado a partir de uma série temporal é uma característica dessa série temporal — a média, o mínimo ou o máximo, por exemplo. Esses valores podem ser calculados usando funções nativas do pacote `package.json` `pandas`. Por exemplo, vamos calcular as médias de todas as séries nos dados de turismo australianos.

```
aus_tourism = pd.read_csv("../data/aus_tourism.csv", parse_dates=["ds"])
mean_df = aus_tourism.groupby("unique_id", as_index=False)["y"].mean()
mean_df.sort_values(by="y").head(10)
```

	id_único	y
158	Ilha Canguru - Austrália do Sul - Outros	0,34
182	MacDonnell-Território do Norte-Outros	0,45

	id_único	y
294	Região Selvagem Ocidental - Tasmânia - Outros	0,48
34	Barkly-Território do Norte-Outros	0,63
86	Vale de Clare - Austrália do Sul - Outros	0,90
38	Barossa-Austrália do Sul-Outros	1.02
154	Kakadu Arnhem-Território do Norte-Outros	1.04
170	Lasseter-Território do Norte-Outros	1.14
298	Wimmera-Victoria-Outros	1,15
183	MacDonnell-Northern Territory-Visiting	1.18

Here we see that the series with least average number of visits was “Other” visits to Kangaroo Island in South Australia.

Rather than compute one feature at a time, it is convenient to compute many features at once. A common short summary of a data set is to compute five summary statistics: the minimum, first quartile, median, third quartile and maximum. These divide the data into four equal-size sections, each containing 25% of the data. The `statistics` function can be used to compute them.

```
summary_stats = tsf.tsfeatures(aus_tourism,
    freq=4, features=[tsf.statistics], scale=False)
summary_stats[["unique_id", "min", "p25", "median", "p75", "max"]].head(10)
```

	unique_id	min	p25	median	p75	max
0	Adelaide Hills-South Australia-Business	0	0	1	4	29
1	Adelaide Hills-South Australia-Holiday	0	6	9	14	36
2	Adelaide Hills-South Australia-Other	0	0	1	2	9
3	Adelaide Hills-South Australia-Visiting	1	9	12	17	81
4	Adelaide-South Australia-Business	69	134	153	177	242
5	Adelaide-South Australia-Holiday	108	135	154	172	224
6	Adelaide-South Australia-Other	26	44	54	63	107
7	Adelaide-South Australia-Visiting	137	179	206	229	270
8	Alice Springs-Northern Territory-Business	1	9	13	18	34
9	Alice Springs-Northern Territory-Holiday	3	17	32	45	77

4.2 ACF features

Autocorrelations were discussed in Section 2.8. All the autocorrelations of a series can be considered features of that series. We can also summarise the autocorrelations to produce new features; for example, the sum of the first ten squared autocorrelation coefficients is a useful summary of how much autocorrelation there is in a series, regardless of lag.

We can also compute autocorrelations of the changes in the series between periods. That is, we "difference" the data and create a new time series consisting of the differences between consecutive observations. Then we can compute the autocorrelations of this new differenced series. Occasionally it is useful to apply the same differencing operation again, so we compute the differences of the differences. The autocorrelations of this double differenced series may provide useful information.

Another related approach is to compute seasonal differences of a series. If we had monthly data, for example, we would compute the difference between consecutive Januaries, consecutive Februaries, and so on. This enables us to look at how the series is changing between years, rather than between months. Again, the autocorrelations of the seasonally differenced series may provide useful information.

We discuss differencing of time series in more detail in Section 9.1.

Setting `features=[acf_features]` computes a selection of the autocorrelations discussed here. It will return six or seven features:

- the first autocorrelation coefficient from the original data;
- the sum of squares of the first ten autocorrelation coefficients from the original data;
- the first autocorrelation coefficient from the differenced data;
- the sum of squares of the first ten autocorrelation coefficients from the differenced data;
- the first autocorrelation coefficient from the twice differenced data;
- the sum of squares of the first ten autocorrelation coefficients from the twice differenced data;
- For seasonal data, the autocorrelation coefficient at the first seasonal lag is also returned.

When applied to the Australian tourism data, we get the following output.

```
acf_feat = tsf.tsfeatures(aus_tourism, freq=4, features=[tsf.acf_features])
acf_feat.head(10).iloc[:, :5]
```

	unique_id	x_acf1	x_acf10	diff1_acf1	diff1_acf10
0	Adelaide Hills-South Australia-Business	0.071	0.13	-0.580	0.41
1	Adelaide Hills-South Australia-Holiday	0.131	0.31	-0.536	0.50

	unique_id	x_acf1	x_acf10	diff1_acf1	diff1_acf10
2	Adelaide Hills-South Australia-Other	0.261	0.33	-0.253	0.32
3	Adelaide Hills-South Australia-Visiting	0.139	0.12	-0.472	0.24
4	Adelaide-South Australia-Business	0.033	0.13	-0.520	0.46
5	Adelaide-South Australia-Holiday	0.046	0.37	-0.343	0.61
6	Adelaide-South Australia-Other	0.517	1.15	-0.409	0.38
7	Adelaide-South Australia-Visiting	0.068	0.29	-0.394	0.45
8	Alice Springs-Northern Territory-Business	0.217	0.37	-0.500	0.38
9	Alice Springs-Northern Territory-Holiday	-0.007	2.11	-0.153	2.11

4.3 STL Features

The STL decomposition discussed in Chapter 3 is the basis for several more features.

A time series decomposition can be used to measure the strength of trend and seasonality in a time series. Recall that the decomposition is written as

$$y_t = T_t + S_t + R_t,$$

where T_t is the smoothed trend component, S_t is the seasonal component and R_t is a remainder component. For strongly trended data, the seasonally adjusted data should have much more variation than the remainder component. Therefore $\text{Var}(R_t)/\text{Var}(T_t + R_t)$ should be relatively small. But for data with little or no trend, the two variances should be approximately the same. So we define the strength of trend as:

$$F_T = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)}\right).$$

This will give a measure of the strength of the trend between 0 and 1. Because the variance of the remainder might occasionally be even larger than the variance of the seasonally adjusted data, we set the minimal possible value of F_T equal to zero.

The strength of seasonality is defined similarly, but with respect to the detrended data rather than the seasonally adjusted data:

$$F_S = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)}\right).$$

A series with seasonal strength F_S close to 0 exhibits almost no seasonality, while a series with

strong seasonality will have F_S close to 1 because $\text{Var}(R_t)$ will be much smaller than $\text{Var}(S_t + R_t)$.

These measures can be useful, for example, when you have a large collection of time series, and you need to find the series with the most trend or the most seasonality. These and other STL-based features are computed using `features=[stl_features]`.

```
stl_feat = tsf.tsfeatures(aus_tourism, freq=4, features=[tsf.stl_features])
stl_feat.head(10).iloc[:, :5]
```

	unique_id	nperiods	seasonal_period	trend	spike
0	Adelaide Hills-South Australia-Business	1	4	0.460	0.000322
1	Adelaide Hills-South Australia-Holiday	1	4	0.531	0.000105
2	Adelaide Hills-South Australia-Other	1	4	0.590	0.000053
3	Adelaide Hills-South Australia-Visiting	1	4	0.487	0.000580
4	Adelaide-South Australia-Business	1	4	0.462	0.000097
5	Adelaide-South Australia-Holiday	1	4	0.578	0.000017
6	Adelaide-South Australia-Other	1	4	0.746	0.000024
7	Adelaide-South Australia-Visiting	1	4	0.449	0.000048
8	Alice Springs-Northern Territory-Business	1	4	0.552	0.000052
9	Alice Springs-Northern Territory-Holiday	1	4	0.379	0.000007

We can then use these features in plots to identify what type of series are heavily trended and what are most seasonal.

```
df = (
    stl_feat["unique_id"].str.split("-", expand=True)
    .rename(columns={0: "region", 1: "state", 2: "purpose"})
    .join(stl_feat)
)

fig, axs = plt.subplots(3, 3, figsize=(8, 8))
axs = axs.flatten()
for ax, (state, state_df) in zip(axs, df.groupby('state')):
    sns.scatterplot(x="trend", y="seasonal_strength",
        hue="purpose", edgecolor="none", data=state_df, ax=ax)
    ax.get_legend().remove()
    ax.set(title=state, xlabel="", ylabel="", xlim=(0, 1), ylim=(0, 1))

handles, labels = ax.get_legend_handles_labels()
fig.legend(handles, labels,
    title="Purpose", loc="center left", frameon=False,
```

```
    bbox_to_anchor=(1.02, .5), borderaxespad=0)

fig.supxlabel('Trend')
fig.supylabel('Seasonal Strength')

# Remove any empty subplots
for ax in axs:
    if not ax.lines:
        ax.set_visible(False)
```

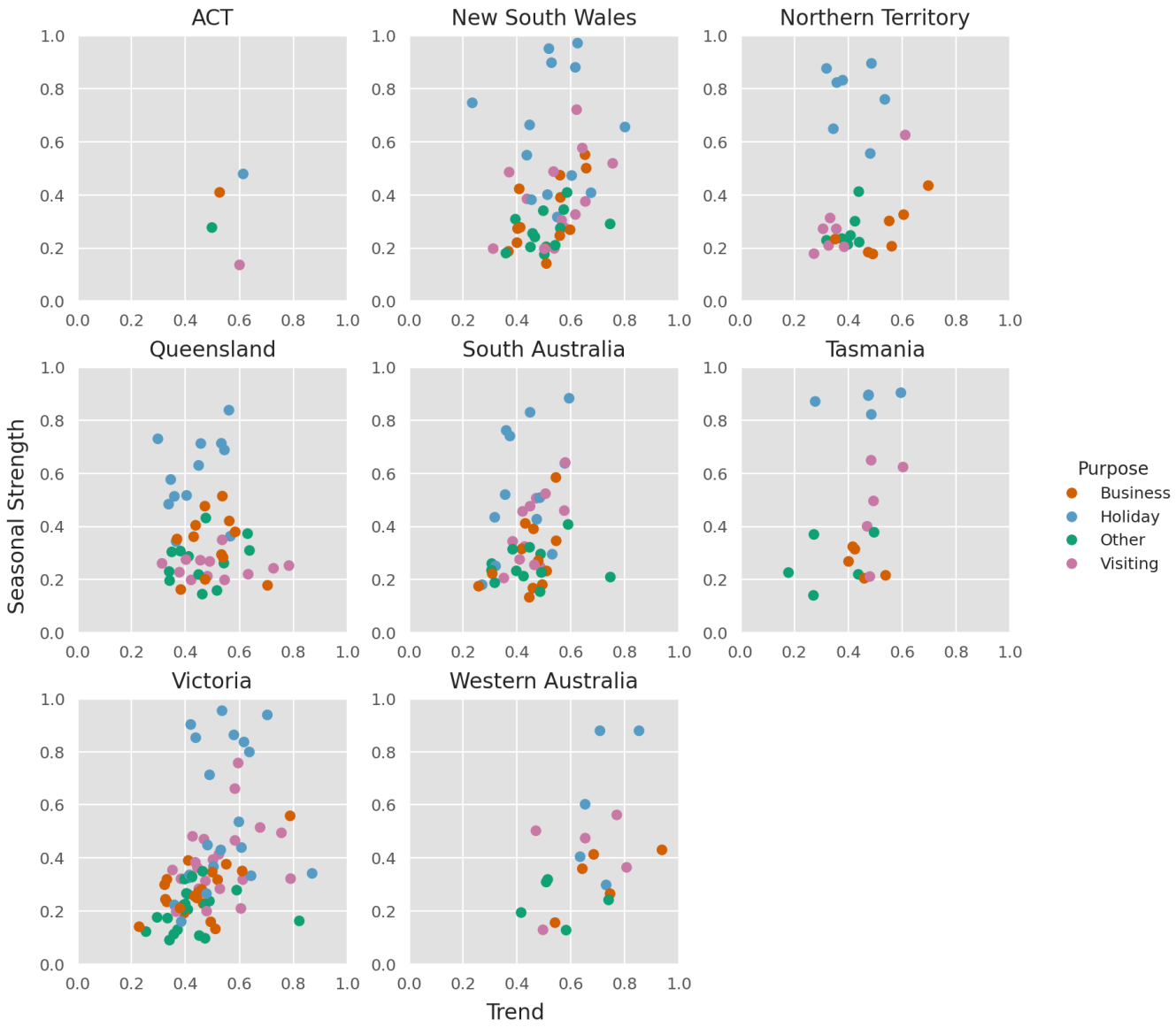

Figure 4.1: Seasonal strength vs trend strength for all tourism series.

Clearly, holiday series are most seasonal which is unsurprising. The strongest trends tend to be in Western Australia and Victoria. The most seasonal series can also be easily identified and plotted.

```
row = stl_feat.loc[lambda x: x["seasonal_strength"].idxmax()]
plot_series(
    aus_tourism, ids=[row["unique_id"]],
    xlabel="Quarter", ylabel="Trips",
    title="New South Wales-Snowy Mountains-Holiday")
```

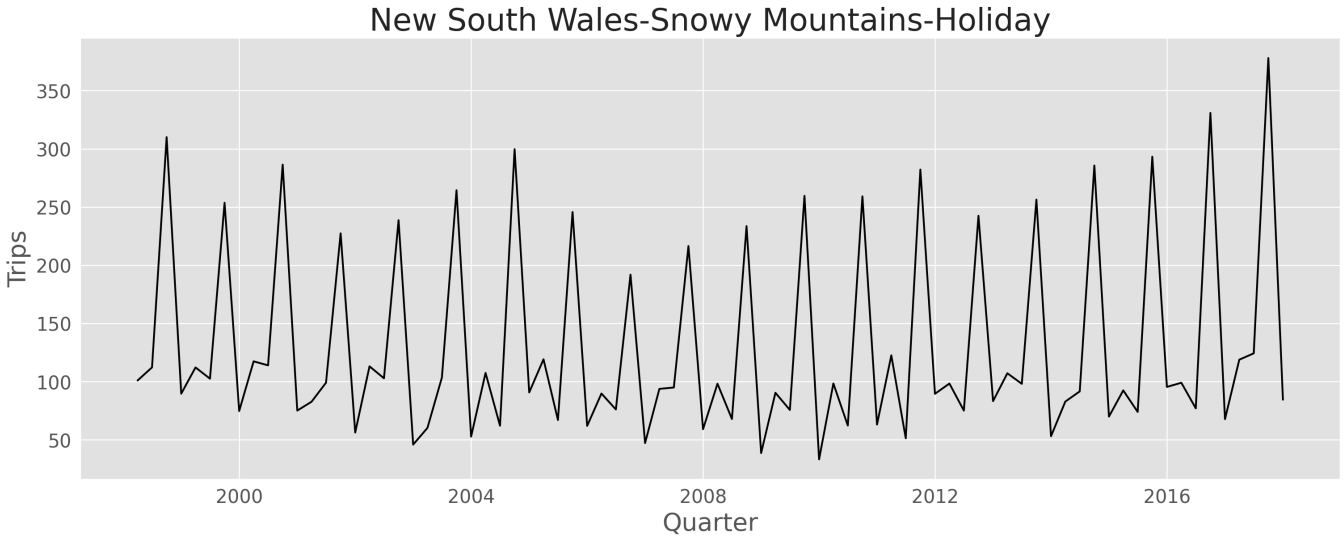

Figure 4.2: The most seasonal series in the Australian tourism data.

This shows holiday trips to the most popular ski region of Australia.

The `stl_features()` function returns several more features other than those discussed above.

- `peak` indicates the timing of the peaks — which month or quarter contains the largest seasonal component. This tells us something about the nature of the seasonality. In the Australian tourism data, if Quarter 3 is the peak seasonal period, then people are travelling to the region in winter, whereas a peak in Quarter 1 suggests that the region is more popular in summer.
- `trough` indicates the timing of the troughs — which month or quarter contains the smallest seasonal component.
- `spike` measures the prevalence of spikes in the remainder component R_t of the STL decomposition. It is the variance of the leave-one-out variances of R_t.
- `linearity` measures the linearity of the trend component of the STL decomposition. It is based on the coefficient of a linear regression applied to the trend component.
- `curvature` measures the curvature of the trend component of the STL decomposition. It is based on the coefficient from an orthogonal quadratic regression applied to the trend component.
- `e_acf1` is the first autocorrelation coefficient of the remainder series.
- `e_acf10` is the sum of squares of the first ten autocorrelation coefficients of the remainder series.

4.4 Other features

Many more features are possible, and the `tsfeatures` library computes only a few dozen features that have proven useful in time series analysis. It is also easy to add your own features

by writing a Python function that takes a univariate time series input and returns a numerical vector containing the feature values.

The remaining features in the `tsfeatures` library, not previously discussed, are listed here for reference. The details of some of them are discussed later in the book.

- `hurst` will calculate the Hurst coefficient of a time series which is a measure of "long memory". A series with long memory will have significant autocorrelations for many lags.
- `feat_spectral` will compute the (Shannon) spectral entropy of a time series, which is a measure of how easy the series is to forecast. A series which has strong trend and seasonality (and so is easy to forecast) will have entropy close to 0. A series that is very noisy (and so is difficult to forecast) will have entropy close to 1.
- `box_pierce` gives the Box-Pierce statistic for testing if a time series is white noise, and the corresponding p-value. This test is discussed in Section 5.4.
- `ljung_box` gives the Ljung-Box statistic for testing if a time series is white noise, and the corresponding p-value. This test is discussed in Section 5.4.
- The kth partial autocorrelation measures the relationship between observations k periods apart after removing the effects of observations between them. So the first partial autocorrelation ($k = 1$) is identical to the first autocorrelation, because there is nothing between consecutive observations to remove. Partial autocorrelations are discussed in Section 9.5. The `feat_pacf()` function contains several features involving partial autocorrelations including the sum of squares of the first five partial autocorrelations for the original series, the first-differenced series and the second-differenced series. For seasonal data, it also includes the partial autocorrelation at the first seasonal lag.
- `n_crossing_points` computes the number of times a time series crosses the median.
- `stat_arch_lm` returns the statistic based on the Lagrange Multiplier (LM) test of Engle (1982) for autoregressive conditional heteroscedasticity (ARCH).
- `guerrero` computes the optimal λ value for a Box-Cox transformation using the Guerrero method (discussed in Section 3.1).

4.5 Exploring Australian tourism data

All of the features included in the `tsfeatures` library can be computed in one line like this.

```
all_features = [
    tsf.acf_features,
    tsf.arch_stat,
    tsf.crossing_points,
    tsf.entropy,
    tsf.flat_spots,
    tsf.heterogeneity,
```

```
    tsf.holt_parameters,
    tsf.lumpiness,
    tsf.nonlinearity,
    tsf.pacf_features,
    tsf.stl_features,
    tsf.stability,
    tsf.hw_parameters,
    tsf.unitroot_kpss,
    tsf.unitroot_pp,
    tsf.series_length,
    tsf.hurst,
]
all_feat = tsf.tsfeatures(aus_tourism, freq=4, features=all_features)
all_feat.head(10)
```

This gives 42 features for every combination of the three key variables (`Region` , `State` and `Purpose`). We can treat this dataframe like any data set and analyse it to find interesting observations or groups of observations.

We've already seen how we can plot one feature against another (Section 4.3). We can also do pairwise plots of groups of features. In Figure 4.3, for example, we show all features that involve seasonality, along with the `Purpose` variable.

```
seasonal_feat = all_feat[
    ["unique_id", "seasonal_strength", "peak", "trough",
     "seas_acf1", "seas_pacf"]
]

df = (
    seasonal_feat["unique_id"].str.split("-", expand=True)
    .rename(columns={0: "region", 1: "state", 2: "Purpose"})
    .join(seasonal_feat)
)
g = sns.pairplot(df, hue="Purpose")
g.fig.set_size_inches(10, 10)
plt.show()
```

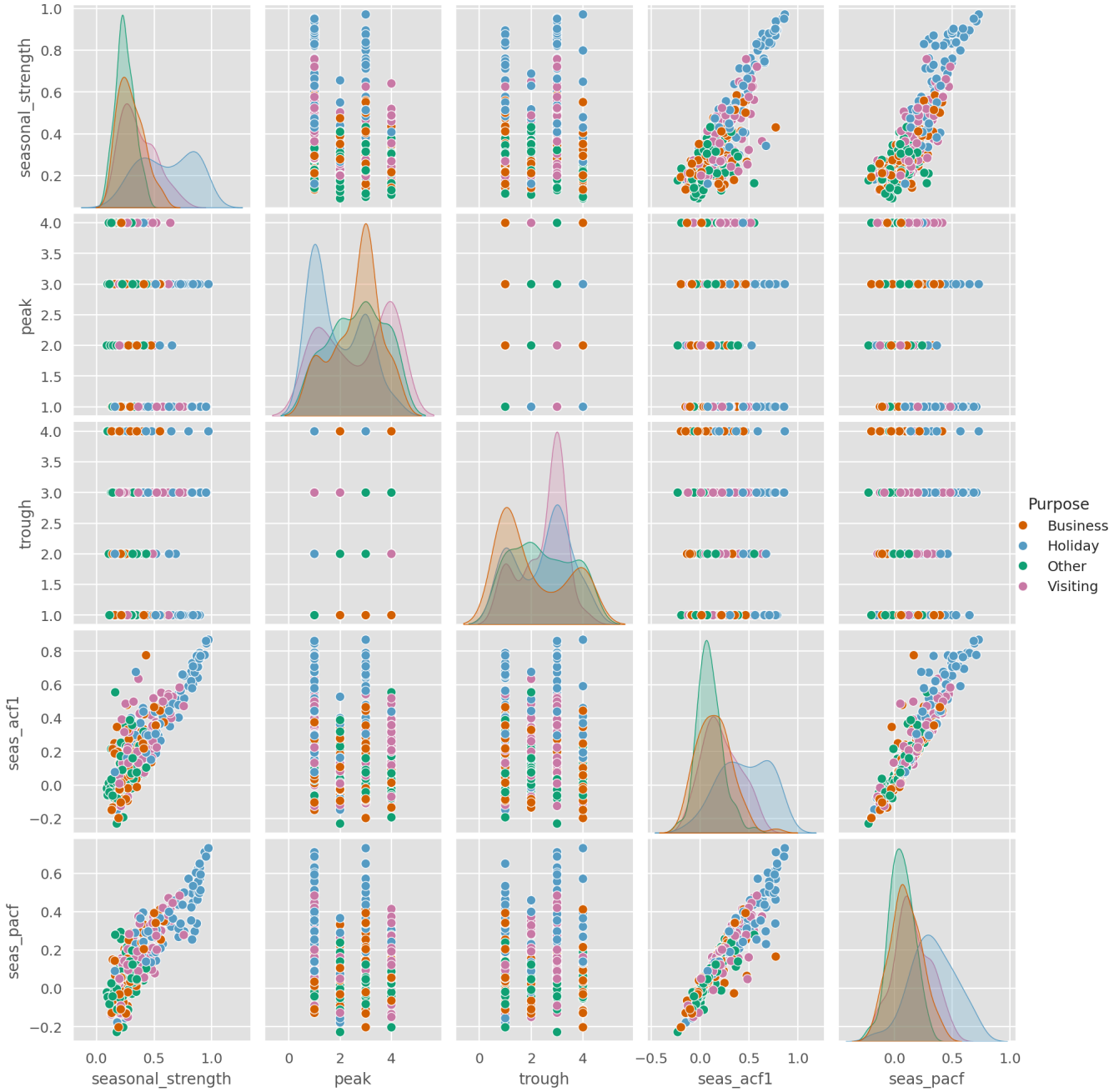

Figure 4.3: Pairwise plots of all the seasonal features for the Australian tourism data.

Here, the `Purpose` variable is mapped to colour. There is a lot of information in this figure, and we will highlight just a few things we can learn.

- The three numerical measures related to seasonality (`seasonal_strength` , `season_acf1` and `season_pacf`) are all positively correlated.
- The bottom left panel and the top right panel both show that the most strongly seasonal series are related to holidays (as we saw previously).
- The bar plots in the bottom row of the `peak` and `trough` columns show that seasonal peaks in Business travel occur most often in Quarter 3, and least often in Quarter 1.

It is difficult to explore more than a handful of variables in this way. A useful way to handle many more variables is to use a dimension reduction technique such as principal components.

This gives linear combinations of variables that explain the most variation in the original data. We can compute the principal components of the tourism features as follows.

```
df = (
    all_feat
    .assign(purpose=lambda x: x["unique_id"].str.split("-").str[-1])
    .dropna(axis='columns')
)
X = df.drop(columns=["unique_id", "purpose"])

pipeline = Pipeline([
    ('scale', StandardScaler()),
    ('pca', PCA(n_components=2)),
])
principal_components = pipeline.fit_transform(X)

pca_df = (
    pd.DataFrame(data=principal_components, columns=["PC1", "PC2"])
    .join(df[["unique_id", "purpose"]])
)

fig, ax = plt.subplots()
sns.scatterplot(x="PC1", y="PC2", s=80,
    hue="purpose", edgecolor="none", data=pca_df, ax=ax)
ax.set(
    title="PCA of Features",
    xlabel="Principal Component 1",
    ylabel="Principal Component 2",
)
ax.get_legend().remove()
handles, labels = ax.get_legend_handles_labels()
fig.legend(handles, labels,
    title="Purpose", loc="center left", frameon=False,
    bbox_to_anchor=(1.02, .5), borderaxespad=0)
plt.show()
```

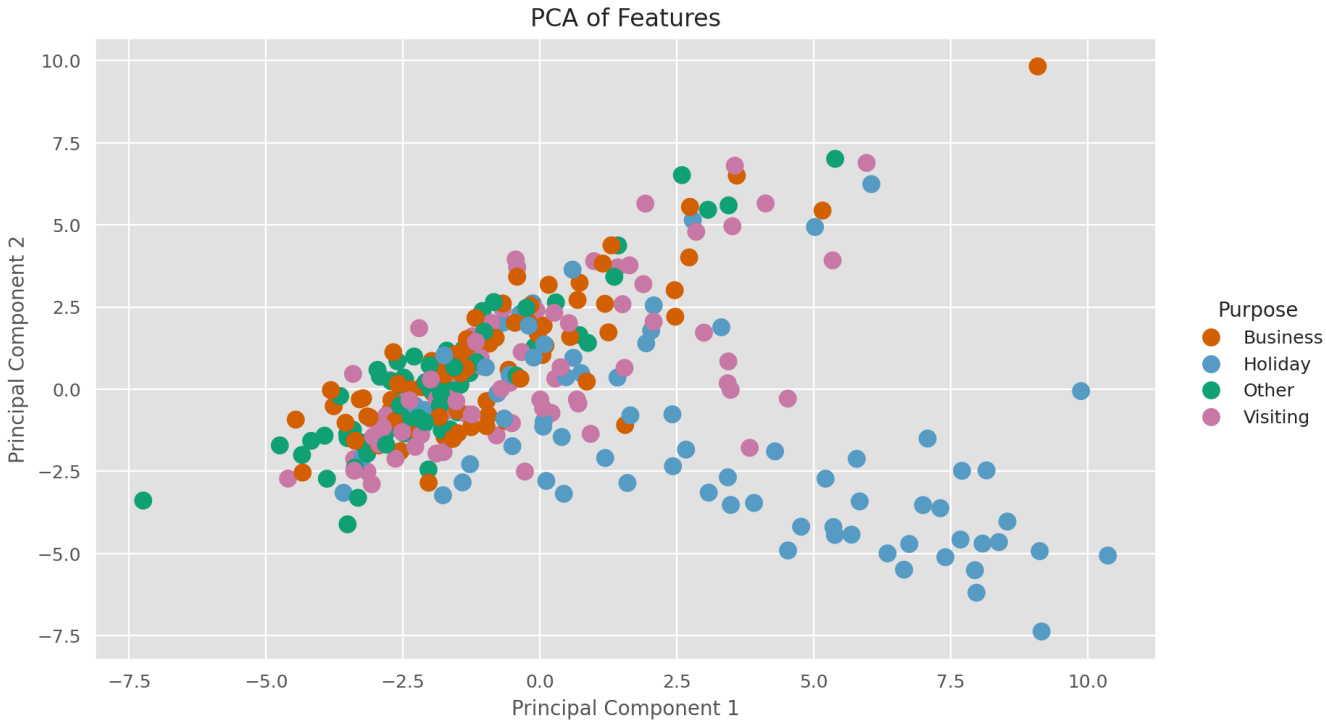

Figure 4.4: A plot of the first two principal components, calculated from the 37 features of the Australian quarterly tourism data.

Each point on Figure 4.4 represents one series and its location on the plot is based on all 37 features. The first principal component (`PC1`) is the linear combination of the features which explains the most variation in the data. The second principal component (`PC2`) is the linear combination which explains the next most variation in the data, while being uncorrelated with the first principal component. For more information about principal component dimension reduction, see Izenman (2008).

Figure 4.4 reveals a few things about the tourism data. First, the holiday series behave quite differently from the rest of the series. Almost all the holiday series appear in the bottom half of the plot, while almost all the remaining series appear in the top half of the plot. Clearly, the second principal component is distinguishing between holidays and other types of travel.

The plot also allows us to identify anomalous time series — series which have unusual feature combinations. These appear as points that are separate from the majority of series in Figure 4.4. There are three that stand out, and we can identify which series they correspond to as follows.

```
lof = LocalOutlierFactor(n_neighbors=10).fit(pca_df[["PC1", "PC2"]])
res = pca_df.assign(score=lof.negative_outlier_factor_)
selected_ids = res.sort_values("score")["unique_id"].head(3)

fig, axs = plt.subplots(3, figsize=(8, 8), sharex=True)
for (ax, unique_id) in zip(axs, selected_ids):
    df = aus_tourism.loc[lambda x: x["unique_id"] == unique_id]
    ax.plot(df["ds"], df["y"], color="k")
    ax.set_title(unique_id, loc="left", size="medium")
```

```
fig.suptitle("Outlying time series in PC space")
fig.supylabel("Trips")
fig.supxlabel("Quarter")
plt.show()
```

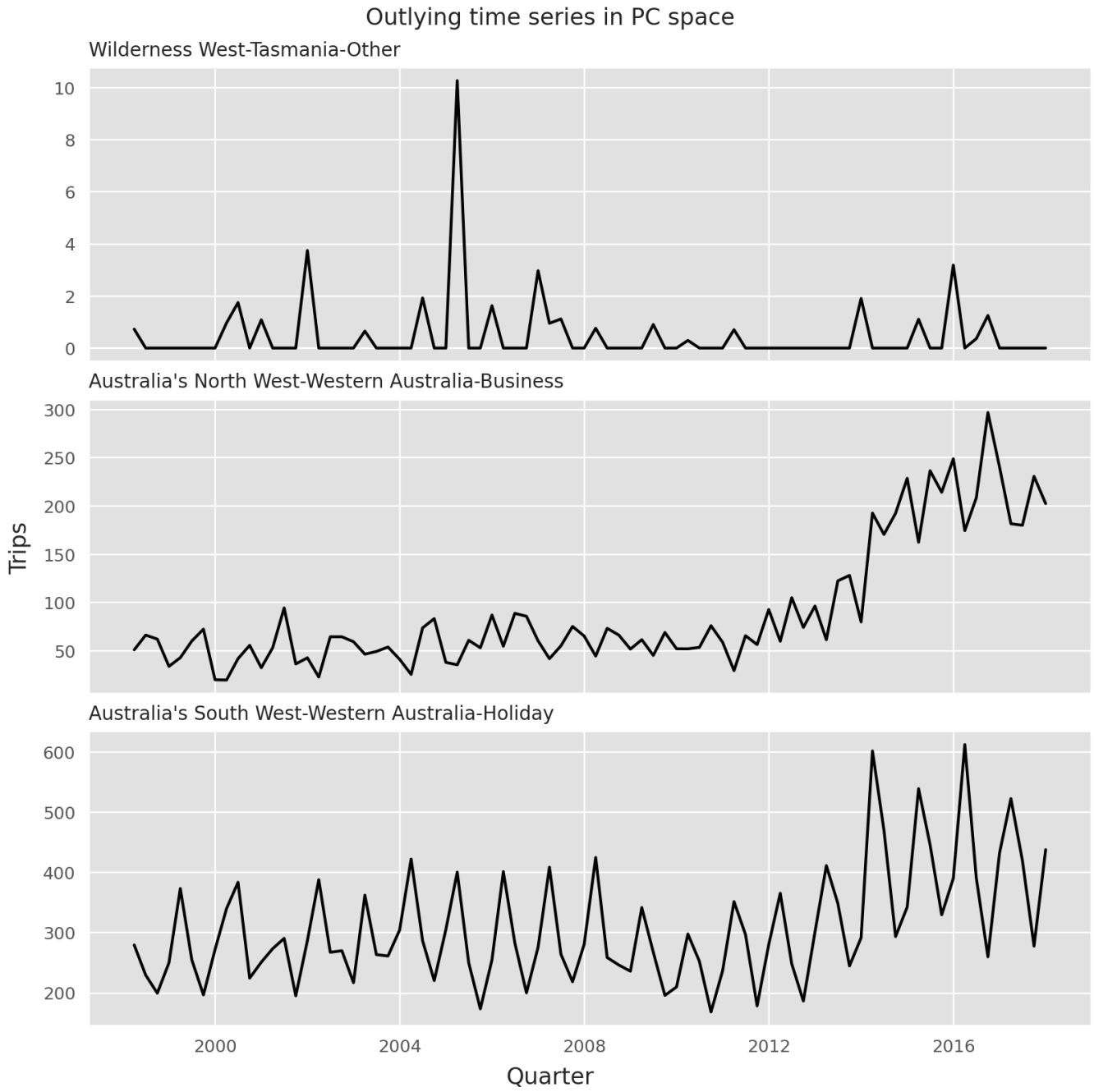

Figure 4.5: Three anomalous time series from the Australian tourism data.

We can speculate why these series are identified as unusual.

- The wild west of Tasmania has very few visitors, but there was an outlier in “Other visits” in 2005.
- The north-western corner of Western Australia is unusual because it shows a strong increase in business visits in the last few years of data (due to mining), but little or no seasonality.
- The south-western corner of Western Australia is also unusual in showing an increase in holiday tourism in the last few years of data, but with a high level of seasonality.

4.6 Exercises

1. Write a function to compute the mean and standard deviation of a time series, and apply it to the `PBS` data. Plot the series with the highest mean, and the series with the lowest standard deviation.
2. Use `sns.pairplot()` to look at the relationships between the STL-based features for the holiday series in the `tourism` data. Change `seasonal_peak_year` and `seasonal_trough_year` to factors, as shown in Figure 4.3. Which is the peak quarter for holidays in each state?
3. Use a feature-based approach to look for outlying series in the `PBS` data. What is unusual about the series you identify as "outliers"?

4.7 Further reading

- The idea of using STL for features originated with Wang, Smith, and Hyndman (2006).
- Os recursos fornecidos pelo `tsfeatures` pacote foram motivados por seu uso em Hyndman, Wang e Laptev (2015) e Kang, Hyndman e Smith-Miles (2017) .
- A exploração de um conjunto de séries temporais usando componentes principais em uma grande coleção de características foi proposta por Kang, Hyndman e Smith-Miles (2017) .

4.8 Módulos e classes utilizados

tsfeatures

- `tsfeatures` Pacote - Para computar características de séries temporais
- `acf_features()`, `stl_features()` funções - Para recursos de autocorrelação e decomposição STL

Previsão de utilidades

- `plot_series()` Utilitário - Para criar visualizações de séries temporais

Publicado pela OTexts usando Quarto . © 2026